Operating Systems Lab — Assignment 7

Q1 — User threads and shared counter

I added `clone(fcn, arg, stack)` and `join()`. The cloned threads share the same address space, but each has its own kernel stack and trapframe mapping. Three threads each try 100,000 increments. In the recorded three-hart run, the counter was:

```
Q1 unsynchronized counter=300000 expected_without_race=300000
```

This run happened to reach 300,000. That does not mean there is no race; unsynchronized increments can still lose updates. Giving each thread its own trapframe mapping fixed the earlier thread startup problem. `join()` cleans up a finished thread without freeing the shared address space.

Q2 — Semaphores and producer-consumer

For Q2, I used `sem_wait()` and `sem_signal()` around each counter increment. The producer and consumer use a circular buffer with five slots. Semaphores track empty slots, full slots, and access to the buffer. Another semaphore keeps each printed log line together.

```
Q2 PASS counter=300000 buffer=0 overflow=0 underflow=0
```

The test processes 50 items. The screenshots show sample produced/consumed items and the final PASS line. Q1 and Q2 both attempted 300,000 increments. Q1 happened to get the full count in this run, while Q2 protects the increment with a semaphore. The buffer finished empty, with no overflow or underflow.

The kernel uses integer semaphore IDs: `sem_init(id, value)`, `sem_wait(id)`, and `sem_signal(id)`. The kernel stores semaphore counts, locks, and initialization status in a fixed array.

Q3 — Deadlock detection and Banker's Algorithm

The program uses Allocation, Need/Request, and Available resource data. Banker's Algorithm checks whether processes have a safe completion order. Deadlock detection checks whether processes can finish using current available resources and their outstanding requests. An unsafe state does not always mean a deadlock is already happening.

The tests cover a safe state, an unsafe state, a valid hypothetical request that makes a safe state unsafe, a no-deadlock matrix, and a deadlocked matrix. A separate pipe test uses two child processes in a circular wait; the parent ends the test so it cannot hang indefinitely.

Recorded Q3 results:

Case	Result
1	Safe sequence: P1, P3, P4, P0, P2
2	Unsafe state; no safe sequence
3	Hypothetical request makes safe state unsafe; request rejected
4	Deadlock detector found no deadlock
5	Deadlock detector identified P0 and P1
Pipe test	Circular wait was bounded and terminated

Q4 — Priority scheduling and aging

I changed the scheduler to choose the RUNNABLE process with the lowest priority number. Priority 1 is highest, priority 20 is lowest, and new processes start at 10. If a RUNNABLE process waits more than 30 ticks, its priority improves by one and its wait counter resets. The scheduler keeps round-robin order for ties.

The test uses four child processes with the same CPU workload and priorities 1, 5, 10, and 20. It records each PID, completion order, elapsed ticks, and the lowest effective priority observed. The recorded run was:

Order	PID	Initial priority	Elapsed ticks	Effective priority
1	4	1	116	1
2	5	5	118	5
3	6	10	118	10
4	7	20	186	17

Priority 20 was observed aging to 19, then 18, then 17. All four children completed. These tick counts are from one run and can change between runs.

Problems and fixes

- Threads initially interfered with each other when returning to user mode because they shared a trapframe mapping. Giving each threadits own trapframe mapping fixed that problem.
- The shared counter needs synchronization in Q2. The semaphore protects each increment, and empty/full/mutex semaphores keep theproducer-consumer buffer within its five slots.
- In Q3, I print the resource state before and after a hypothetical request to show why the request is rejected. The pipe circular-wait test isbounded and terminated by the parent.
- The first Q4 test used different workloads and too many children, so the completion times were hard to compare. I changed it to fourchildren with equal workloads and recorded their results.

Build and test

The recorded checks were `make kernel/kernel`, `make fs.img`, and `git diff --check`. The four user programs were run in xv6: `threadtest`, `synctest`, `deadlock`, and `priotest`.

To reproduce, go to the xv6 source directory and run `make kernel/kernel`, `make fs.img`, then `make CPUS=3 qemu`. At the xv6 prompt, run `threadtest` and `synctest`. Restart with `make CPUS=1 qemu` for `deadlock`, and with `make CPUS=3 qemu` for `priotest`.